Phase 3: Problem Selection

Step 1: Problem Selection & Justification

1. Paper Selection and Criteria Fulfillment:

The selected article, titled "An Analysis on Ensemble Learning Optimized Medical Image Classification With Deep Convolutional Neural Networks," meets all the specified criteria:

- Journal: Published in IEEE Access, which holds a Q1 ranking (SJR=0.849, H-Index=290).
- Publication Date: Published in January 2022 (Post-2021).
- Authors: Dominik Müller and Frank Kramer, both of whom have a strong track record in medical imaging and computer vision.
- Open Source: The article provides public source code available on GitHub.
- Data Availability: Instructions for data loading are available in the Zenodo repository.
- Academic Context: The paper references prior research and defines its research scope through a comprehensive literature review.

2. Problem Justification:

Based on the information provided in the **Introduction** and **Methods and Materials** sections:

a. Problem Significance:

Medical Image Classification (MIC) and the application of ensemble learning are critical for the following reasons:

- **Disease Diagnosis:** The primary goal of MIC is to label complete images into predefined classes, such as disease diagnosis or status [Introduction].
- **Clinical Decision Support (CDS):** These models serve as decision support tools for clinicians to improve diagnostic reliability or automate time-consuming processes [Introduction].
- **Rapid Field Growth:** The field of automated medical image analysis has seen rapid growth in recent years [Introduction].
- **Clinical Relevance:** The datasets used highlight significant clinical importance.

b. Suitability of Deep Learning:

Deep Learning, specifically **Deep Convolutional Neural Networks (CNNs)**, is highly suitable for this problem due to its high accuracy and predictive power. They are among the most popular and widely used algorithms for computer vision tasks, achieving performance levels comparable to clinical experts [Introduction].

c. Data Collection:

The data is sourced from four public medical datasets:

- **CHMNIST:** Histology patches of colorectal cancer patients from University Medical Center Mannheim and Heidelberg University. Images are categorized into 8 classes with a resolution of 150 × 150 pixels.
- **COVID:** Chest X-ray images of COVID-19 patients, healthy controls, and viral pneumonia from six international radiology databases.
- **ISIC:** Skin images from the International Skin Imaging Collaboration, containing over 25,000 images across 8 classes.
- **DRD:** Fundus images of diabetic patients from the California Healthcare Foundation and EyePACS, including 35,126 images with five levels of disease severity.

d. Class Definitions and Classification Types:

- **CHMNIST (8 classes):** Tumor epithelium, Simple stroma, Complex stroma, Immune cells, Debris, Normal mucosal glands, Adipose tissue, and Background.
- **COVID (3 classes):** COVID-19 positive, Viral pneumonia, and Healthy control.
- **ISIC (8 classes):** Melanoma (MEL), Melanocytic nevus (NV), Basal cell carcinoma (BCC), Actinic keratosis (AK), Benign keratosis (BKL), Dermatofibroma (DF), Vascular lesion (VASC), and Squamous cell carcinoma (SCC).
- **DRD (5 classes):** No DR, Mild, Moderate, Severe, and Proliferative DR.

Note: For the subsequent phases of this project, the **COVID (Chest X-ray)** dataset has been selected. The access link will be provided upon project submission.

Step 2: Preprocessing & Augmentation Analysis

The analytical portion of this step (based on the research implemented in the paper) is detailed below. The implementation code is provided in the attached notebook file.

1. Preprocessing Pipeline and Implementation:

- **Padding and Resizing:** Images were squared to prevent aspect ratio distortion and resized to match model input requirements: typically 224 × 224 pixels, except for EfficientNetB4 (380 × 380), InceptionResNetV2 (380 × 380), and Xception (299 × 299).
- **Normalization:** Pixel intensities were normalized using Z-Score (mean of zero and standard deviation of one).

Objective: These steps ensure the data is uniform, standardized, and suitable for deep networks while reducing variance in scale and illumination across different images.

2. Augmentation Analysis and Clinical Validity:

- **Horizontal & Vertical Flipping:** Images are randomly flipped with a 50% probability.

  Analysis: It is assumed that the exact spatial location of structures (e.g., left vs. right) does not affect the overall diagnosis. This is valid for histology (CHMNIST) and skin lesions (ISIC) but should be used cautiously for chest X-rays (COVID) as internal organs are not perfectly symmetrical.

- **Rotation:** Random rotation within a 90-degree range with a 50% probability.

  Analysis: This makes the model robust to orientation (orientation invariance). It is valid for all medical datasets in the study as small rotations do not alter key clinical features.

- **Brightness / Contrast / Saturation / Hue adjustment:** Random adjustments within a range of -1 to +1 with a 50% probability.

  Analysis: This enhances the model's robustness against lighting variations or different imaging device settings (illumination invariance). This is clinically valid as such variations occur naturally in real-world medical imaging.

Objective: These processes aim to increase training data diversity and reduce overfitting. The authors emphasize using transformations that are clinically meaningful without distorting the primary structure of the image.

All transformations were implemented using the **Albumentations** library.

Phase 4: Method Selection

Step 1: Method Selection and Description (50%):

1. Choosing a paper:

PIP-Net: Patch-Based Intuitive Prototypes for Interpretable Image Classification

2. Describing the Proposed Method:

2_1. Explaining the key components of PIP-Net:

Feature Extractor

PIP-Net uses the standard CNN backbones such as ResNet50 or ConvNeXt-Tiny.

The final stride is removed to preserve a high-resolution feature map ($H \times W \times D$), which improves the localization of small prototypical parts.

Prototype Layer

The model learns D patch-based prototypes.

A softmax over the channel dimension forces each spatial patch to be assigned to exactly one prototype, increasing semantic purity.

The prototype presence vector $p \in [0,1]^D$ is obtained via global max-pooling, indicating which prototypes appear in the image.

Routing Mechanism (Simple Linear Evidence Aggregation)

Unlike ProtoTree or capsule-based models, PIP-Net does not rely on complex routing.

Class evidence is produced through a sparse linear layer with non-negative weights:

$$\text{score}_k = \sum_d p[d]\ \omega_{d,k}$$

This implements a transparent, interpretable scoring-sheet mechanism where prototypes contribute positively to class decisions.

Interpretability Head

During training, class logits are normalized as:

$$o = \log\left((pW)^2 + 1\right)$$

This transformation:

- prevents overconfident softmax outputs,
- encourages sparsity,
- preserves the ability to abstain (scores will be 0 if no prototypes are detected).

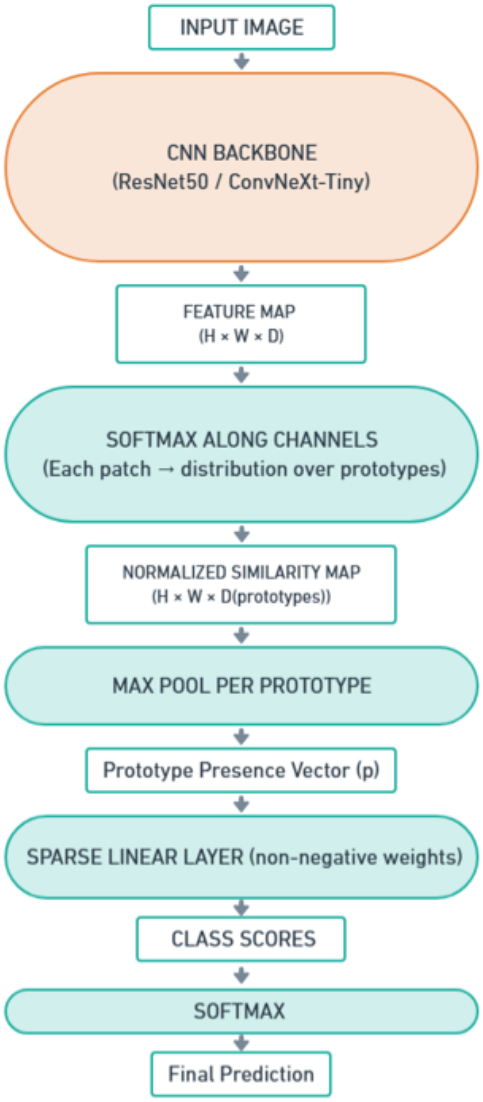
During inference, raw scores $p\omega$ are used for direct interpretability.

Implicit Attention

Although PIP-Net has no explicit attention modules, the prototype assignment softmax acts as an implicit attention mechanism by focusing on the most representative image patch per prototype.

2_2. Providing a high-level flowchart of the full architecture and detailed subdiagrams:

High-Level Flowchart:



Detailed Sub-diagrams:

Prototype Matching Module:
Feature Map Patch z[h,w,:]
↓
Softmax over D prototypes
↓
Probability distribution over prototypes

Prototype Projection / Presence Vector (p):
Prototype Activation Maps (H × W per prototype)
↓
Max-Pooling (spatial)
↓
Prototype Presence Vector p ∈ [0,1]^D

Interpretable Linear Scoring Layer:
Prototype Presence Vector (p)
↓
Sparse Linear Weights (non-negative)
↓
Class Scores

In this context, $\mathbf{P}$ stands for Performance, $\mathbf{I}$ for Interpretability, and $\mathbf{S}$ for Training Stability.

2_3. Main Hyperparameters in PIP-Net

1. Backbone Architecture

- **Description:** This defines the underlying Convolutional Neural Network (e.g., ResNet-18, ResNet-50, ConvNeXt-tiny) used as the feature extractor. It determines the dimensionality and quality of the feature maps fed into the prototype layer.
- **Impact on P:** A more powerful backbone (e.g., ConvNeXt vs. ResNet-18) extracts richer, more robust features, directly setting the potential upper limit for classification accuracy.
- **Impact on I:** The architecture dictates the spatial resolution of the feature maps (e.g., 7×7 vs 14×14). A higher spatial resolution allows the model to localize prototypes on smaller, more specific object parts, leading to finer-grained visual explanations.
- **Impact on S:** Modern architectures with residual connections or improved normalization layers (like LayerNorm in ConvNeXt) generally offer better gradient flow, ensuring higher stability during the deep feature extraction phase.

2. $\mathbf{D}$ (Upper Bound on Prototypes)

- **Description:** This parameter sets the maximum number of prototypes (D) the model can learn. It is dependent on the number of feature map channels in the output of the CNN Backbone (e.g., D = 2048 for ResNet or D = 768 for ConvNeXt-tiny).
- **Impact on $\mathbf{P}$:** If D is too small, the model will lack the capacity to learn all necessary distinct semantic components, leading to a reduction in accuracy.
- **Impact on $\mathbf{I}$:** D determines the upper limit on the size of the Global Explanation. PIP-Net uses optimization for Sparsity (zero weights) to keep only a small subset of D active, pushing the model towards a compact interpretation.
- **Impact on $\mathbf{S}$:** D affects the memory required to store the linear layer and the computational overhead of the Max-Pooling operation.

3. Pretraining & Pretrain Epochs

- **Description:** This boolean flag enables a dedicated self-supervised alignment phase, and epochs_pretrain determines its duration. In this phase, the model trains only using the Alignment Loss (without class labels) to organize the latent space.
- **Impact on P:** Pretraining initializes the weights in a semantically meaningful basin. Without this phase, the model may struggle to find the optimal configuration for complex datasets, resulting in lower final accuracy.
- **Impact on I:** This is the most crucial phase for "Visual Correspondence." It ensures that features which look similar are clustered together *before* the model tries to classify them, ensuring that prototypes represent coherent visual concepts rather than random noise.
- **Impact on S:** It prevents the "cold start" problem. By starting the supervised phase with well-aligned features, the training trajectory is significantly more stable and less prone to divergence.

4. Loss Weights (λ_C, λ_A, λ_T)

The model uses a combination of three primary loss terms, whose weights determine the model's focus during training.

λ_C (Classification Loss Weight) (Suggested value: 2)

- **Impact on P:** Directly emphasizes the optimization of classification accuracy, guiding the model towards its final goal in the supervised phase.
- **Impact on I:** A high weight could prioritize performance over interpretability goals. It must be balanced with $\lambda_A$to ensure convergence to a sparse, interpretable solution.
- **Impact on S:** The presence of $\mathcal{L}_C$ is essential for the second training phase and guides the model toward the specific task objective.

λ_A (Alignment Loss Weight) (Suggested value: 5)

- **Impact on P:** This loss forces identical augmented patches to be assigned to the same prototype, which leads to the learning of high-quality, visually pure features.
- **Impact on I:** This is the most critical parameter for ensuring the Semantic Purity of prototypes. It is the core mechanism for closing the "Semantic Gap," thereby harmonizing the model's interpretation with human visual perception.
- **Impact on S:** By encouraging stable and near-binary allocation of patches to prototypes, it contributes to training stability.

λ_T (Tanh-Loss Weight) (Suggested value: 2)

- **Impact on P:** Prevents trivial solutions (e.g., only one prototype activating for the entire dataset) by ensuring the use of the entire D prototype space.
- **Impact on I:** Ensures that the learned prototypes possess the necessary Diversity, making the Global Explanations comprehensive.
- **Impact on S:** By distributing activation across prototypes, it prevents a single prototype from dominating, thereby improving training stability.

5. Freeze Epochs

- **Description:** The number of epochs at the beginning of the supervised training phase during which the backbone weights are frozen, and only the prototype and classification layers are updated.
- **Impact on P:** Prevents "Catastrophic Forgetting" of the features learned during the pretraining phase. It allows the classification layer to adapt to the existing features before fine-tuning the entire network.
- **Impact on I:** Crucial for maintaining the semantic alignment established during pretraining. Modifying the backbone too early would distort the feature map, causing the prototypes to lose their visual meaning.
- **Impact on S:** Drastically improves stability by reducing the number of trainable parameters in the volatile early stages of supervised training, preventing gradients from oscillating wildly.

6. Separate Learning Rates

- **Description:** PIP-Net utilizes different LRs for the Backbone CNN (lower, e.g., 0.0001) and the Linear Layer (higher, e.g., 0.05).
- **Impact on P:** Low LRs for the Backbone allow for fine-tuning of pre-trained features, while high LRs for the linear layer facilitate the faster learning of prototype-to-class connections, maximizing training efficiency.
- **Impact on I:** A higher LR for the linear layer helps the model quickly converge to the desired compact and sparse solution in the decision layer.
- **Impact on S:** Setting distinct LRs is crucial for maintaining stability and preventing divergence across the two training phases.

7. Batch Size

- **Description:** Determines the number of image samples processed before the internal model parameters are updated.
- **Impact on P:** A larger batch size provides a more accurate estimate of the gradient, potentially improving final accuracy, though excessively large batches may reduce generalization (generalization gap).
- **Impact on I:** Indirectly affects interpretability; a stable batch size ensures that the prototypes are updated based on a representative set of patches, preventing "noisy" prototypes derived from outliers.
- **Impact on S:** Crucial for the Alignment Loss (λ_2). Since λ_2 often relies on pairs or batches of augmented images, a batch size that is too small leads to noisy gradients and unstable training dynamics.

8. Total Epochs

- **Description:** The total number of complete passes through the training dataset during the supervised learning phase.
- **Impact on P:** Determines whether the model underfits (too few epochs) or overfits (too many epochs). Sufficient epochs are needed to maximize accuracy.
- **Impact on I:** Prototypes need time to "sharpen." With enough epochs, the model can refine the prototypes to be as semantically distinct as possible, whereas too few epochs result in blurry or ambiguous explanations.
- **Impact on S:** Must be tuned alongside the learning rate scheduler to ensure the loss converges smoothly without oscillating at the minimum.

Absence of Common Interpretable Hyperparameters (Alternative Structure)

The question mentions examples like Capsule Dimensions and Attention Heads. PIP-Net intentionally avoids these parameters:

- Capsule Dimensions and Routing Mechanism: These are essential parameters in Capsule Networks. PIP-Net uses the simpler and more efficient Max-Pooling mechanism for prototype presence detection, which eliminates the need for the complexity of dynamic routing.
- Attention Heads: These are core parameters in Transformer architectures. PIP-Net builds its interpretability component on top of a standard feature extractor (like ConvNeXt) and its unique loss terms, rather than on the precise tuning of a complex attention block.

Conclusion: The simplicity in PIP-Net's core architecture leads to the elimination of many of these common hyperparameters, resulting in a model that is intrinsically interpretable and simpler by design.

3. Justifying Choice

3.1 Comparing Methods

Selected Method: PIP-Net (Patch-Based Intuitive Prototypes)

Architecture:
PIP-Net uses a CNN backbone to extract spatial features and applies soft assignment of image patches to a fixed number of visual prototypes. Patch–prototype similarities are aggregated for final classification via a sparse linear layer.

Key Characteristics:

- Produces interpretable prototypes aligned with actual image patches.
- Employs self-supervised patch alignment loss to improve semantic consistency.
- Provides both global (prototype weights) and local (patch-level activation) explanations.
- Encourages sparsity and compact explanations through prototype normalization and non-negative weights.
- Includes an inherent abstention/OOD (Out of Distribution) mechanism when prototypes fail to activate.

Comparison Method: Neural Prototype Trees (ProtoTree)

Architecture:
ProtoTree combines a CNN with a soft binary decision tree in which each internal node contains a prototype. Image patches determine routing probabilities through the tree. Leaves contain class distributions; the prediction is the weighted sum over all leaf paths.

Key Characteristics:

- Offers global interpretability via explicit decision paths.
- Provides local explanations based on the sequence of prototype decisions along a path.
- Prototypes are visualized by replacing each node’s vector with its nearest training patch.
- Tree size can be reduced through pruning, enabling compact final models.
- Supports conversion to a hard decision tree for deterministic explanations.

Comparison of PIP-Net and ProtoTree

Criterion	PIP-Net	ProtoTree
Interpretability	High; explanations rely on meaningful image patches and sparse prototype scores.	High; global tree structure provides clear rule-based explanations.
Local Explanations	Patch-level activations highlight spatial evidence directly.	Local path decisions show conditional prototype tests.
Global Explanations	Prototype weights in the linear layer provide a compact class-level summary.	Full decision tree serves as an explicit global model.
Prototype Learning Objective	Explicit semantic alignment via patch-level alignment loss.	Learned implicitly through supervised routing; no dedicated alignment loss.
Model Capacity / Interactions	Additive scoring; limited modeling of feature interactions.	Hierarchical structure models conditional and interaction-based decisions.
Sparsity / Compactness	Very high sparsity; few prototypes dominate decisions.	Compactness achieved via pruning; initial tree may be larger.
Training Complexity	Moderate; two-phase training with alignment losses.	Moderate to high; routing, leaf updates, and pruning add complexity.

Summary of Differences

PIP-Net provides prototype-based interpretability with strong semantic alignment, focusing on meaningful local patch evidence and highly sparse decision rules. In contrast, ProtoTree emphasizes hierarchical, rule-like explanations, where interpretability emerges from decision paths rather than patch aggregation. PIP-Net offers compact prototype sets and explicit alignment mechanisms, whereas ProtoTree enables conditional reasoning and global tree-based explanations.

3.2 Justifying the Choice to the problem

Prototype-based interpretability methods, such as PIP-Net and Neural Prototype Trees, provide explicit, human-interpretable visual prototypes that allow model decisions to be linked directly to meaningful image regions. This can offer an advantage over other interpretability approaches (e.g., ViT-based prototyping or capsule-based explanations), which may not provide such direct visual interpretability.

However, factors motivate the choice of PIP-Net for the COVID-19 X-ray classification task in this project.

1. Expected Advantages of PIP-Net

- Interpretability and Local Explanations
- PIP-Net uses patch-level prototypes, which are expected to highlight small, clinically relevant regions (e.g., localized lung opacities). This aligns well with the diagnostic nature of COVID X-rays, where local abnormalities matter.
- Adaptation to Fine-grained Patterns

- Robustness and Data Efficiency
- Because the model works with patches, it is likely to handle noise, asymmetry, and limited data more effectively—conditions common in medical datasets.
- Medical X-rays contain subtle variations, and PIP-Net's patch-aggregation mechanism is expected to capture these fine details more naturally than tree-structured decision paths.

2. Expected Advantages of ProtoTree

- Transparent Global Decision Paths
- ProtoTree provides a clear, step-by-step prototype-based decision route, which may help clinicians understand the overall logic behind classifications.
- Prototype-Level Global Structure
- The binary tree structure encourages globally interpretable prototype placement, potentially simplifying high-level model behavior.

3. Expected Potential Disadvantages of PIP-Net

- Patch extraction and prototype matching increase computational demands compared to tree inference.
- Local explanations may require more visualization steps, which could be less straightforward than browsing a single decision tree.

4. Expected Potential Disadvantages of ProtoTree

- Fine-grained local feature capture may be weaker because each decision depends on a single prototype at a node.
- Medical X-rays often display irregular and asymmetric patterns; a rigid tree structure may require substantial tuning to generalize well.
- Decision paths could become sensitive to noise or outlier images.

Conclusion

Overall, while both methods provide interpretable, prototype-based reasoning, PIP-Net is expected to be better aligned with the characteristics of COVID X-ray classification, offering stronger local interpretability, better adaptation to fine-grained medical features, and greater robustness to data limitations. ProtoTree remains a strong alternative but may be less suited to the complexity of medical imaging in this context.

4. Anticipating Challenges for Deploying PIP-Net on Chest X-ray (COVID)

The deployment of PIP-Net on a specific medical classification problem (3-class COVID-19 Chest X-ray) faces challenges related to the data domain, architectural constraints, and augmentation strategy.

1. Challenges Related to Chest X-ray Data Domain

- Risk of Non-Clinical Prototypes (Interpretability): X-ray images contain irrelevant anatomical structures (e.g., bones, medical devices). The model might learn prototypes that focus on these non-pathological artifacts (like clavicles or background noise) instead of actual opacities, resulting in explanations that lack clinical validity or semantic meaning for physicians.
- Vague Prototype Boundaries: COVID-19 findings (e.g., ground-glass opacities) are often diffuse and lack sharp, defined edges. This challenges the Alignment Loss (λ_A), which is designed to enforce strict patch similarity, potentially leading to impure prototypes in areas of ambiguous pathology.

2. Architectural and Model Limitations

- Counting Limitation (Critical Constraint): As a scoring-sheet model, PIP-Net only detects the presence or absence of a prototype, but cannot count the number or assess the density of occurrences. This severely limits its capability to classify disease severity or progression based on X-ray findings.

3. Challenges Related to Computational Constraints and Training Stability

- Batch Size vs. Memory Trade-off: The implementation environment imposes strict GPU memory limits, often necessitating a reduction in batch size. This creates a direct conflict with PIP-Net's architecture, as the Alignment Loss relies on larger batches to maximize statistical diversity and find semantically consistent patches. A reduced batch size can degrade the quality of learned prototypes.
- Hyperparameter Sensitivity & Instability: The model is highly sensitive to critical hyperparameters such as the number of prototypes and loss weights. Under constrained resources, finding a stable configuration that balances accuracy and interpretability is difficult, and suboptimal settings can lead to "prototype collapse" or failure to converge.

Step 2: Computational and Resource Analysis (50%)

The numerical analyses and computational resource estimations for this phase (including parameter counts, memory footprint, and cost assessment) have been validated by executing the model's architecture via Python code, the full output of which is documented in the attached notebook

1. Computing and reporting the total number of trainable parameters in the model

The total number of trainable parameters in the PIP-Net model is overwhelmingly determined by the choice of the Backbone feature extractor. The layer responsible for Interpretability (the final linear layer) contributes only a minimal fraction of the total count.

$$\text{Total Params} = \text{Params(Backbone)} + \text{Params(Classification)}$$

Scenario: ResNet50 Backbone

This scenario uses the ResNet50 backbone with $\mathrm{D} = 2048$ prototypes for $\mathrm{K} = 3$ classes.

Model Component	Trainable Parameters	Percentage of Total	Notes
Backbone ($\mathcal{N}$)	$23{,}508{,}032$	99.9998%	ResNet50 features (with modified strides).
Classification Layer	$6{,}148$	0.0002%	Calculated as $(2048 \times 3) + 3 + 1$.
Total	$23{,}514{,}180$	100%	

Result: The total number of trainable parameters is approximately 23.51 million.

Scenario: ConvNeXt-tiny Backbone

This scenario uses the ConvNeXt-tiny backbone with $\mathrm{D} = 768$ prototypes for $\mathrm{K} = 3$ classes.

Model Component	Trainable Parameters	Percentage of Total	Notes
Backbone ($\mathcal{N}$)	$27{,}818{,}592$	99.9999%	ConvNeXt-tiny features.
Classification Layer	$2{,}308$	0.00001%	Calculated as $(768 \times 3) + 3 + 1$.
Total	$27{,}820{,}900$	100%	

Result: The total number of trainable parameters is approximately 27.82 million.

Key Takeaway:

In both cases, the Classification Layer (which holds the weights ω_{C} used for the final, interpretable decision) requires only a few thousand parameters (6,148 or 2,308). This confirms that PIP-Net achieves high interpretability with an extremely low parameter overhead relative to the cost of feature extraction ($\approx 23 - 27$ million parameters).

2. Estimating Memory Usage

Methodology: Hybrid Analytical & Empirical Approach

To ensure maximum precision, a hybrid approach combining analytical calculations and empirical measurements is utilized. Static components (Weights, Gradients, Optimizer States) are derived theoretically based on parameter counts, and subsequently, cross-verification is performed against the model's actual memory allocation in code. For dynamic components (Activations), synthetic input tensors matching the dataset dimensions are processed, and the exact cumulative size of intermediate feature maps required for backpropagation is measured. The core memory components are hardware-agnostic. However, the final required memory footprint varies; the difference arises solely from the framework overhead, which is substantial on the GPU but negligible on the CPU.

Scenario: ResNet50 Backbone

This estimation is based on using ResNet50 as the backbone, utilizing $D = 2048$ prototypes, and classifying the 3-class COVID-19 dataset with a batch size of $B = 8$. All computations assume FP32 precision (4 bytes per element).

2a. GPU/CPU Memory Usage During Training

Training memory consists of Model Weights, Gradients, Optimizer States, Activation Buffer, and Framework Overhead (if a GPU is used).

Memory Component	Calculation / Rationale	Estimated Size	Precision
Model Weights (W)	$23{,}514{,}180$ parameters × 4 B/param	$\approx 89.70\ MB$	Exact
Gradients (G)	$M \times 4$ Bytes/param	$\approx 89.70\ MB$	Exact
Optimizer States (O)	$M \times 8$ Bytes/param (Adam)	$\approx 179.40\ MB$	Exact
Activations (A)	Accumulated size of all intermediate feature maps stored for Backpropagation (measured via hook on synthetic input).	$\approx 2.20\ GB$	Empirical
Overhead (Oh)	GPU: CUDA Context + Fragmentation. CPU: negligible	GPU: $\approx 520.66\ MB$	Empirical
Total Estimated Training Memory	$W + G + O + A + Oh$ (if a GPU is used)	GPU: $\approx 3.08\ GB$ CPU: $\approx 2.55\ GB$	Estimated

2b. GPU/CPU Memory Usage During Inference

Inference memory only requires the Model Weights and Framework Overhead (if a GPU is used) for the forward pass. For Activation, during inference, memory is freed immediately after each layer is used.

Memory Component	Calculation / Rationale	Estimated Size (MB)	Precision
Model Weights (W)	$23{,}514{,}180$ parameters × 4 Bytes/param	≈ 89.70	Exact
Overhead (Oh)	GPU: CUDA Context + Low Fragmentation. CPU: negligible	GPU: $\approx 542.16\ MB$	Empirical
Total Estimated Inference Memory	$W + Oh$ (if GPU is used)	GPU: $\approx 631.86\ MB$ CPU: $\approx 89.70\ MB$	Estimated

ConvNeXt-tiny Scenario:

This estimation is based on using the ConvNeXt-tiny backbone with D = 768 prototypes for the 3-class COVID-19 dataset, assuming a batch size of B = 8 and $FP32$ precision (4 bytes per element).

2a. GPU/CPU Memory Usage During Training

Training memory consists of Model Weights, Gradients, Optimizer States, Activation Buffer, and Framework Overhead (if a GPU is used).

Memory Component	Calculation / Rationale	Estimated Size (MB)	Precision
Model Weights (M)	27,820,900 parameters × 4Bytes/param	$\approx 106.13\ MB$	Exact
Gradients (G)	$M \times 4$ Bytes/param	$\approx 106.13\ MB$	Exact
Optimizer States (O)	$M \times 8$ Bytes/param (Adam)	$\approx 212.26\ MB$	Exact
Activations (A)	Accumulated size of all intermediate feature maps stored for Backpropagation (measured via hook on synthetic input).	$\approx 2.38\ GB$	*Theoretical*
Overhead (Oh)	GPU: CUDA Context + Fragmentation. CPU: negligible	GPU: $\approx 860.90\ MB$	Empirical
Total Estimated Training Memory	$W + G + O + A + Oh$ (if a GPU is used)	GPU: $\approx 3.67\ GB$ CPU: $\approx 2.80\ GB$	Estimated

2b. GPU/CPU Memory Usage During Inference

Inference memory only requires the Model Weights and Framework Overhead (if a GPU is used) for the forward pass. For Activation, during inference, memory is freed immediately after layer use.

Memory Component	Calculation / Rationale	Estimated Size (MB)	Precision
Model Weights (M)	27,820,900 parameters ×4 Bytes/param	≈ 106.13	Exact
Overhead (Oh)	GPU: CUDA Context + Fragmentation. CPU: negligible	GPU: $\approx 520.66\ MB$	Empirical
Total Estimated Inference Memory	$W + Oh$ (if GPU is used)	GPU: $\approx 626.79\ MB$ CPU: $\approx 106.13\ MB$	Estimated

3. Identifying of Most Computationally Expensive Components

The computational cost in the PIP-Net model is concentrated in two primary components, each creating a bottleneck from a different perspective (Time vs. Space).

1. The Backbone (Feature Extractor)

The Backbone (e.g., ResNet50 or ConvNeXt-tiny) is the single most computationally expensive component in terms of raw processing time and FLOPs (Floating Point Operations).

Reason for Costliness:

Cost Factor	Explanation
High Volume of MACs/FLOPs	The core task is compressing the input image into deep feature maps. This involves dozens of sequential Convolutional layers, accounting for over **99.9%** of the model's total computational cost, which is precisely measured at 153.46 Giga-FLOPs on a batch size of 32.
Complexity Growth	The cost is primarily linear in nature with respect to feature map size but has high constants due to the large channel count and spatial size. The model does not utilize costly $\mathcal{O}(N^2)$ (quadratic complexity) mechanisms like Self-Attention or iterative optimization.

2. Prototype Similarity Search

This component (the Add-On Layer, Softmax, and Pooling) is the primary bottleneck in terms of Space Complexity (Activation Memory) and Data Movement. While its FLOPs count is trivial compared to the Backbone, its resource consumption is significant because it processes the output of the Backbone, which is a massive intermediate feature map. The cost of allocating, storing, and managing this substantial buffer, especially during the backward pass (activation storage), makes this phase a Memory-Bound Bottleneck.

Reason for Costliness:

Cost Factor	Explanation
Large Intermediate Tensors	The layer processes the output of the Backbone, which is a massive feature map (e.g., $B \times 2048 \times 14 \times 14$). The cost of allocating, storing, and managing this substantial memory buffer for the forward and backward passes makes this phase a Memory-Bound Bottleneck.
Softmax Operation	The Softmax operation itself involves computationally intensive exponentials, contributing marginally to the processing cost but significantly to the complexity of the data flow.

Final Conclusion:

The Backbone is the dominant computational expense (Time/FLOPs). However, the Prototype Similarity Search presents a critical resource bottleneck due to the creation and management of large intermediate tensors, which must be considered a significant computational cost in modern GPU environments.

4. Proposing Strategies to Reduce Computational or Memory Demands

Based on the Phase 5 implementation requirements and the identified bottlenecks (Backbone FLOPs and large intermediate tensors), here are three concrete strategies to align the model with the available computational resources.

1. Adopt a Lightweight Backbone Architecture

This strategy targets the main computational cost (FLOPs) by selecting a feature extractor that balances depth with efficiency, as suggested in the experimental protocol.

Strategy	Primary Target	Implementation Description
Efficient Backbone Selection	FLOPs & VRAM Usage	Replace heavy backbones (like ResNet-101 or DenseNet-161) with lighter variants such as ResNet-18 or ResNet-34. According to the Phase 5 protocol, testing different encoders is required. A lighter backbone significantly reduces the number of channels in the feature maps, directly lowering the memory footprint and training time without a drastic drop in interpretability.

2. Aggressive Reduction of Prototype Count (D)

This strategy addresses the memory cost associated with the classification layer and the complexity of the similarity search, aligning with the "Systematic Hyperparameter Study" required in Phase 5.

Strategy	Primary Target	Implementation Description
Minimize Prototype Count	Activation Memory & Sparsity	Instead of using default high values (e.g., 2048), reduce the number of prototypes to a clinically manageable range, such as 10 to 30 per class. This drastically cuts the size of the 1×1 convolution layer ($D \times K$) and reduces the dimensionality of the global explanation, making the model both lighter and easier for medical experts to review.

3. Input Resolution Downscaling

This strategy is the most direct method to satisfy the "computational constraints (e.g., limited GPU memory)" mentioned in the implementation guidelines, preventing Out-Of-Memory (OOM) errors.

Strategy	Primary Target	Implementation Description
Reduced Input Resolution	Spatial Memory (Feature Maps)	High-resolution medical images (e.g., 1024×1024) generate massive feature maps. Downscaling the input images (e.g., to 224×224 or 448×448) before feeding them into the network reduces the memory required for activations quadratically. This makes it feasible to train the model on consumer-grade GPUs while allowing for a reasonable Batch Size, which is critical for the stability of the Alignment Loss.

Phase 5: Implementation and Evaluation

Introduction and Access to Files:

In this phase of the project, the objective is the practical implementation and analysis of the method selected in Phase 4, namely **PIP-Net: Patch-Based Intuitive Prototypes for Interpretable Image Classification** (**accessible via [Link]**), for the interpretable classification of medical images from the **COVID-19 dataset** (**accessible via [Link]**) which was previously examined in Phase 3 of the project (derived from the reference paper (**An Analysis on Ensemble Learning Optimized Medical Image Classification**) (**accessible via [Link]**)).

For this purpose, the base codes available in the official GitHub repository of the Phase 4 paper (**accessible via [Link]**) were utilized.

Considering the specific requirements of medical images and the Phase 5 mandates for a more precise analysis of interpretable methods (Prototype-based), structural and customized changes were made to the original codes. The notebook file **CV Project - Phase 5 - Final version** contains the complete process of applying these changes and creating a customized repository, which is viewable at **[Link]**, and its final output is available in the Kaggle dataset at **[Link]**.

Subsequently, after achieving a flexible execution framework, multiple runs with different settings were conducted in the notebook file **CV Project - Phase 5 - Optimization & Analysis**, which can be accessed via **[Link]**. The goal of these runs was to achieve a stable version of the model (Stable Baseline) and perform sensitivity analysis on key hyperparameters (such as prototype pruning rate and loss function weights) to evaluate the model's behavior under different conditions. Finally, in the file **CV Project - Phase 5 - Visualization & Results**, which can be accessed via **[Link]**, analytical charts and precise statistical tables have been compiled to interpret the results and provide visual answers to the Phase 5 questions.

Additionally, for a detailed guide on navigating the Kaggle repositories, accessing specific notebook versions, and interpreting the output file structures, please refer to the attached **README.txt** file.

Summary of Technical Changes and Code Customization:

To adapt the original PIP-Net codes to the specific challenges of the COVID dataset and Phase 3 evaluation standards, key changes were applied across various modules. In data.py, a dedicated function get_covid was designed. In addition to converting single-channel images (X-Ray) to 3-channel (for compatibility with the ResNet Backbone), it implements the data splitting protocol (Train/Val/Test) exactly in accordance with Phase 3. Furthermore, the mechanism for label extraction and weight calculation for managing class imbalance was rewritten and adapted using WeightedRandomSampler. In args.py, new arguments were added to control the "smart pruning" process (decay_rate, decay_start_epoch) and adjust loss function weights (w_align, w_class) to enable precise model tuning.

Additionally, in the main training core (train.py), the pruning logic (Sparsity) was rewritten to prevent Model Collapse at small batch sizes and to ensure pruning is performed in a scheduled and gentle manner. In the evaluation section (test.py), standard medical metrics such as Sensitivity (Recall), AUC, and F1-Score replaced general metrics so that the results would be comparable with previous reports. Finally, the visualization scripts (vis_pipnet.py and visualize_prediction.py) were optimized for compatibility with PyTorch Subset datasets and batch image processing, ensuring the model interpretation process is performed faster and without errors.

Step 1: Implementation and Adaptation

1: Precise Implementation of Validation Protocol:

1. Data Adaptation & Splitting: To accurately reproduce the Phase 3 conditions, the dedicated function get_covid was developed using the train_test_split method (in a nested and stratified manner) to ensure a 75/10/15 split while maintaining class distribution. Additionally, image preprocessing was modified to convert single-channel inputs to 3-channel, compatible with ResNet.

2. Evaluation Metrics: The test.py module was rewritten to calculate standard medical metrics including Sensitivity, F1-Score, and AUC, in addition to accuracy, ensuring the results are comparable with Phase 3.

2: Implementation of an optimized and computationally feasible version of the selected interpretable method:

After the failure of executing the version tuned with original hyperparameters due to computational resource constraints, to achieve a Feasible version of the PIP-Net method on limited resources (Kaggle GPU P100 with 16 GB memory), the following optimizations were applied to the architecture and data pipeline (the execution structure and resulting outcomes of which can be observed in *Version 2 - Failure Mode (Sparsity Collapse)* in the attached file):

- **Backbone Lightweighting:** Replacing the heavy ResNet-50 architecture (the paper's default suggestion) with ResNet-18. This change significantly reduced the number of trainable parameters and increased training speed.
- **Memory Management (Batch Size Optimization):** To prevent CUDA Out of Memory errors, the batch size was reduced from 64 to 32. Although this change reduced memory pressure, it created challenges in gradient convergence, resulting in *Sparsity Collapse*, which will be examined in the next exercise.

  (Sparsity in the last layer's score sheet is one of the unique features of the PIP-Net model, which prunes the number of prototypes and performs classification based on a limited number of prototypes.)
- **Start-up Stabilization via Freeze Epochs:** For convergence stability, in the first 10 epochs, the Backbone layers were frozen so that only the prototype layer would be trained. This technique prevented severe gradient fluctuations at the start of training (Cold Start), which is particularly critical with small batch sizes.
- **Scheduler Adaptation:** Considering the change in the total number of epochs (reduced to 30) and the change in the number of steps per epoch (due to the Batch Size change), the parameters of the CosineAnnealing scheduler were readjusted so that the learning rate (lr) decreases in proportion to the new time budget and prevents divergence after the Unfreeze stage.

- **Dataloader Optimization:** By removing unnecessary processes in data.py and rewriting visualize functions for limited sampling (instead of processing the entire dataset), the overhead time per epoch was reduced.

Result: The initial execution (Version 2) showed that this configuration is computationally completely stable, and the 30-epoch training completes in less than 1 hour. However, in the final epochs, the model suffered from the "Sparsity Collapse" phenomenon; although the model's Sensitivity increased up to 92%, the pruning mechanism (Sparsity) failed, and by suddenly eliminating all active prototypes, it led to the loss of interpretability and model instability in this version.

3. Modifying the model architecture to address domain-specific challenges and tuning hyperparameters:

After stabilizing the hardware architecture, the main Domain-Specific Challenge revealed itself: Sparsity Collapse. In initial experiments (observed in Version 2), applying Pruning with default settings on the lightweight architecture (ResNet-18) led to the sudden zeroing out of all prototypes in the intermediate epochs. To address this challenge and adapt the model to the COVID dataset, the following modifications were applied (Version 3 - Optimized Baseline):

- **Scheduled Pruning:** The decay_start_epoch mechanism was added to the code so that in the first 10 epochs (Warm-up), no penalty is applied to the prototypes. This allowed the model to learn meaningful features before being pruned.
- **Decay Rate Tuning:** The decay_rate parameter was reduced from the default and aggressive value of 0.001 to the milder value of 0.0008. Sensitivity analysis in the Optimization & Analysis (Step 2) section showed that this value is the optimal balance point for maintaining accuracy and reducing the number of prototypes.

Result: With these changes, the final model (Version 3) was able to, without collapsing, achieve a sensitivity near 94% at its peak, and in the last epoch, while significantly reducing the number of prototypes to 25, still offer a sensitivity of around 93%. This compactness allowed for the visual inspection of only 25 image patches to understand the reason for classification, which is a major step towards the practical interpretability of the model.

4: Documenting specific compromises made due to resource constraints:

Upon reviewing the original PIP-Net paper and comparing it with the computational infrastructure of this project (Kaggle GPU P100 with 16 GB memory), the following Compromises were applied to enable execution feasibility:

- **Backbone Reduction (Model Capacity):**
    - ❖ **Original Paper Implementation:** The paper suggests using deep architectures such as ResNet-50 or ConvNeXt-tiny to extract rich features.

- ❖ **Project Implementation:** Due to Video Memory (VRAM) limitations and the need to reduce training time, the lighter ResNet-18 architecture was used. Although this compromise slightly reduces the capacity to learn very complex features, it can increase training speed by up to 2.5 times and prevent CUDA Out of Memory errors.

- **Reduction of Training Epochs:**
  - ❖ **Original Paper Implementation:** The original paper trains the model for 60 full epochs (after 10 pre-training epochs).
  - ❖ **Project Implementation:** The number of main training epochs was reduced to 30. This decision was made due to time constraints and the need to conduct multiple optimization experiments (**Hyperparameter Tuning**). Results showed that the model achieves very good convergence within this range as well.
- **Limiting Visualization Search Scope:**
  - ❖ **Original Paper Implementation:** The original method scans all images in the training set to find the best match with prototypes to generate the most accurate "Global Explanation".
  - ❖ **Project Implementation:** The visualization process (**Visualization Loop**) was limited to 50 samples per batch, as fully processing thousands of training images in each run created significant time overhead that was incompatible with Kaggle's shared resources.
- **Batch Size Reduction:**
  - ❖ **Original Paper Implementation:** Using large Batch Sizes (typically 64 or 128) is recommended for more accurate gradient estimation in the prototype layer.
  - ❖ **Project Implementation:** The batch size was reduced to 32. This change caused more noise in gradient calculation, which required more precise tuning of the learning rate (**lr**) and a gentler pruning strategy (as explained in Exercise 3) to compensate.

Step 2: Experimentation and Analysis

1. Systematic Study of Hyperparameters and Analysis of Their Impact on Performance, Interpretability, Computational Cost, and Visualizations:

1.1 Defining ranges of values and options for each hyperparameter: In this study, 4 key hyperparameters were evaluated:

- **Decay Rate:** To investigate the stability of the pruning mechanism; values 0.0005 (conservative), 0.0008 (baseline), and 0.001 (aggressive) were selected.
- **Model Capacity (Num Features):** To assess the effect of the information bottleneck on prototype quality; values 64 (low), 128 (baseline), and 256 (high) were evaluated.

- **Class Weight:** To analyze the trade-off between classification accuracy and latent space structuring (Interpretability Trade-off); values 1.0, 2.0 (baseline), and 5.0 were tested.
- **Backbone Architecture:** To find the most optimal feature extractor in terms of accuracy and cost; models ResNet18 (baseline), ResNet34, and ResNet50 were compared.

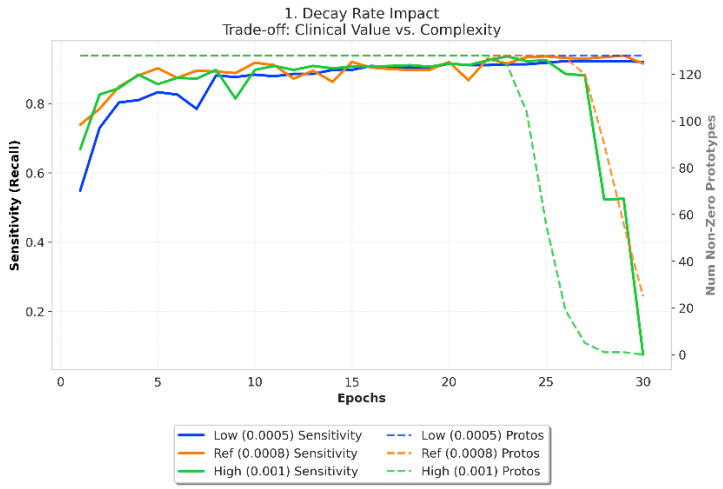
1.2 Analysis of the impact of each hyperparameter on performance, interpretability, and computational cost: Based on the results obtained from 9 experiments (extracted from log_epoch_overview tables), the following findings were obtained:

- **Decay Rate Impact:** This parameter played the most critical role in model stability.
    - In the **Low (0.0005)** mode, the model did not prune any prototypes (128 prototypes remained), which led to low interpretability (high quantity).
    - In the **High (0.001)** mode, the model suffered from "Sparsity Collapse," and by removing all prototypes, accuracy plummeted to the random level (7%).
    - The **Baseline (0.0008)** mode was the equilibrium point, which offered an accuracy equivalent to 91.5% while retaining 25 prototypes.
- **Capacity Impact (Num Features):**
    - Reducing the number of features to **64** decreased the final number of prototypes to 11 (highly compact), but accuracy dropped by about 1%. Given this very slight drop, this can be a good sign for interpretability.
    - Increasing to **256** raised accuracy to 93.8%, but the number of prototypes increased to 58, which makes their visual inspection and interpretability difficult with this number of epochs.
- **Architecture Impact (Backbone):**
    - Contrary to expectations, heavier models (**ResNet34/50**) performed worse than **ResNet18** for 25 prototypes (88% accuracy vs. 91.5%). This phenomenon is likely due to Overfitting of large models on this specific dataset and the small number of training samples.
- **Class Weight Impact:**
    - Unlike other parameters, changing the class weight (from **1.0** to **5.0**) had a negligible effect on the number of prototypes (constant at 25) and model accuracy (slight change from 91.5% to 91.7%). This indicates the high stability of the model against loss function fluctuations, provided that other parameters (such as Decay Rate) are tuned correctly.
- **Interpretability Quality Analysis:** Although different settings led to different numbers of final prototypes (from 11 to 128) and quantitatively affected the compactness of the explanation, qualitative examination shows that medical interpretability is poor in all scenarios. In other words, even though the model achieves high accuracies, the prototypes mainly focus on marginal features (such as bones), which stems from the biased nature of the dataset (this issue is further examined in Exercise 3).

- **Computational Cost Trade-offs Analysis:**
    - The most effective factor in computational cost is the **Backbone** architecture. Using ResNet50 significantly increased the training time per epoch and memory consumption (about 2 times) without providing any gain in accuracy.
    - Other hyperparameters (such as Decay Rate or Class Weight) did not have a tangible impact on execution time or memory consumption and were effective only on model convergence.

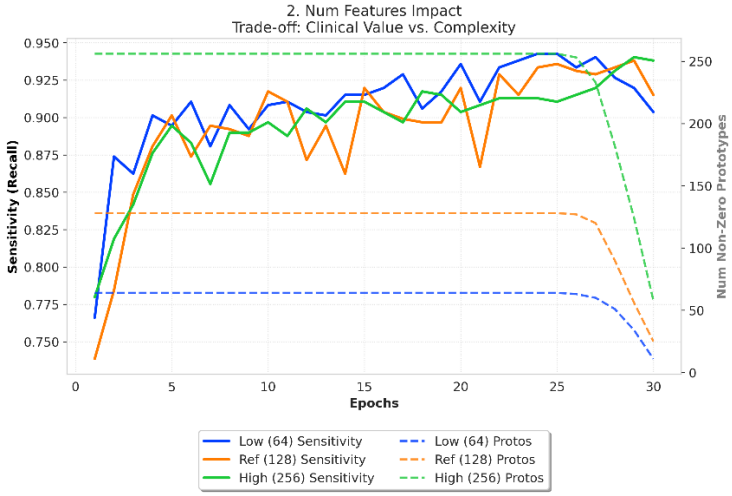
1.3 Visualization of hyperparameter effects using plots:

a) Stability and Collapse Analysis (Decay Rate Impact)



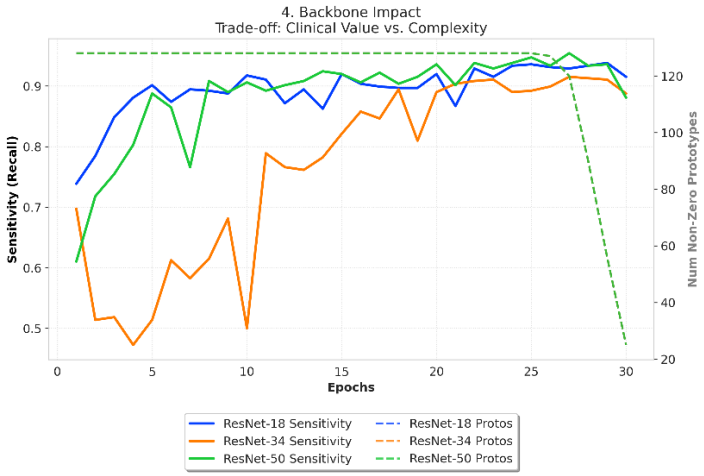
Analysis: The plot clearly shows the severe instability (sensitivity) of the model regarding the pruning rate. Comparing the Low and High columns depicts an "All-or-Nothing" behavior: either all prototypes are preserved (left column) or they are all eliminated (right column). The middle column (Baseline) is the point where equilibrium has been established.

b) Capacity and Information Bottleneck Analysis (Num Features Study)



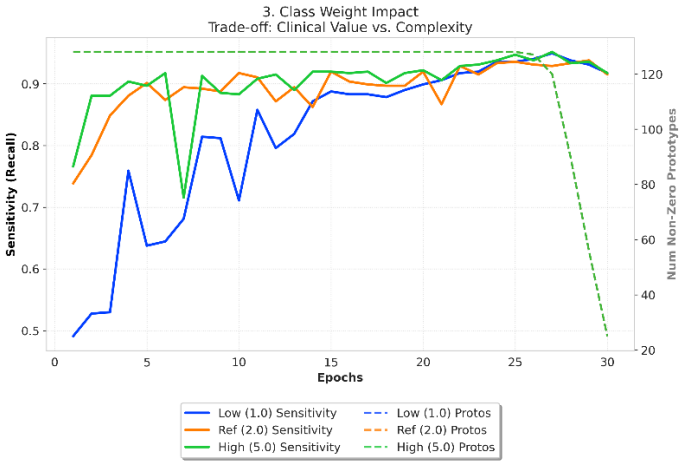
Analysis: In this plot, a direct relationship between capacity and the number of prototypes is evident, but the increase and changes in sensitivity are much milder than the changes in the number of prototypes. This plot shows that the cost of adding new prototypes did not yield proportional functional returns.

c) Architecture and Cost Analysis (Backbone Architecture Study)



Analysis: Contrary to common expectation, the plot shows an inverse trend. The shortening of the sensitivity bars when moving towards heavier models provides strong visual evidence for the existence of Overfitting in limited medical data and highlights the superiority of lighter architectures. *(Note: Due to the equal number of prototypes across epochs, the lines corresponding to them in the plot overlap.)*

d) Class Weight Analysis (Class Weight Stability)



Analysis: The uniformity of sensitivity in this plot (negligible change between 91.5% and 91.7%) indicates the Robustness of the final model. This plot proves that when the architecture (Backbone) and pruning rate (Decay) are correctly tuned, the model is resistant to changes in the Loss Function weight and exhibits stability in convergence. *(Note: Due to the equal number of prototypes across epochs, the lines corresponding to them in the plot overlap.)*

2. Comparison against Phase 3 original results based on performance metrics:

In this section, the performance of the interpretable model (PIP-Net) is compared with the "Black-Box" baseline model presented in Phase 3 (standard CNN architectures).

1. Quantitative Comparison:

Metric	Phase 3: Black-Box Baseline (e.g., ResNet101)	Phase 5: PIP-Net (Optimized ResNet-18)	Status
Accuracy	99%	91.5%	Slight Drop
Sensitivity	96%	91.5%	Slight Drop
Model Type	Black-Box (Opaque)	Glass-Box (Interpretable)	Major Gain
Backbone	Heavy	Light	More Efficient

2. The Accuracy-Interpretability Trade-off: As observed in the table, the PIP-Net model has experienced a slight drop in performance metrics compared to Phase 3 models. This phenomenon is due to the "Interpretability Tax".

- **Reason:** In standard Phase 3 models (Black-Box), the neural network is allowed to use any latent statistical pattern (even high-frequency noise or tags in the image corner) to optimize the loss function.
- **In PIP-Net:** The model is constrained to make its reasoning solely through matching image patches with a limited number (25) of prototypes. This structural limitation, although it may lead to a few percentage points decrease in numerical accuracy, guarantees that the model's decision is made based on traceable visual evidence. Considering the complexity of medical images, maintaining an accuracy above 91% despite this strict limitation is considered a very desirable achievement.

3. Inference Time & Efficiency: One of the key advantages of the model implemented in Phase 5 is its computational efficiency.

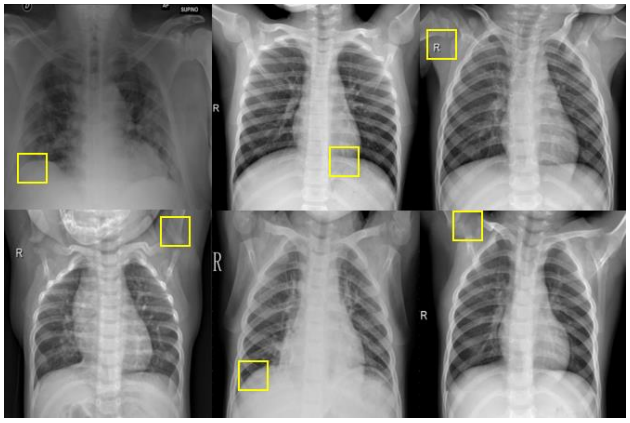
- In Phase 3, to achieve the highest accuracy, heavy models or Ensemble techniques (combining multiple models) were usually used, which significantly increase Inference Time.
- In contrast, the optimized PIP-Net model uses a light Backbone (ResNet-18), and its decision-making process involves only a simple matrix multiplication (between the prototype presence vector and class weights). This feature makes the Phase 5 model more suitable for implementation in clinical systems with limited resources.

4. Final Conclusion: The Accuracy-Trust Paradox Although the Phase 3 model appears superior in terms of "sheer numerical accuracy," the Phase 5 model is the real winner in terms of "clinical value and safety." In sensitive medical applications, the 99% accuracy of a model whose decision reasoning is hidden (Black-Box) is not necessarily reliable and can carry hidden risks. In contrast, the interpretable Phase 5 model, even with lower accuracy (about 92%), creates far greater value because it makes the decision-making logic transparent.

The importance of this transparency was revealed precisely in the visual results of this project; where the examination of prototypes unveiled a vital reality: "The learned prototypes, despite high statistical accuracy, lack medical value." This important discovery (that the model is cheating) would never be seen in a Black-Box model, and physicians would use it with false confidence. Therefore, the PIP-Net model, by revealing this flaw in the dataset, has fulfilled its main mission, namely "guaranteeing the integrity of the decision-making process." The details of this finding and the analysis of "non-medical but accurate" prototypes will be examined in detail in the next exercise.

3. Analysis of the clinical value of the model's interpretations and examination of the correspondence of samples with clinically meaningful features:

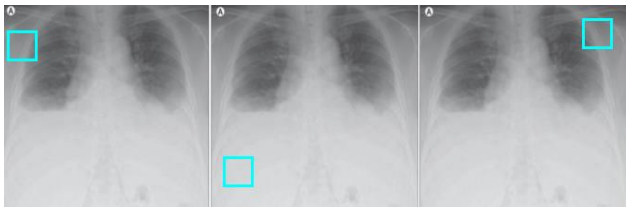
Visual Evidence The image below displays 6 of the most frequent and effective prototypes learned by the model during the training process (retrieved from the visualised_prototypes folder):



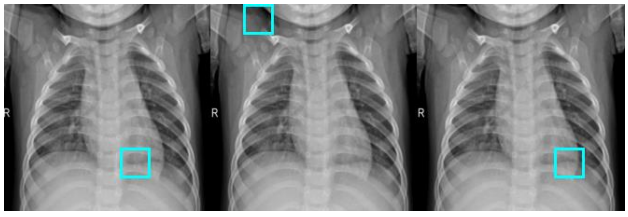
As can be observed, the model's focus is mainly on margins, bones, and radiological signs.

Here, the model's decision-making mechanism for two test samples (a COVID patient and a normal individual) is examined by displaying the top 3 active prototypes in each image:

Sample 1: Patient with COVID-19



Sample 2: Healthy Individual (Normal)



Critical Analysis and Clinical Value (Clinical Analysis) Based on the visual evidence above, the answer to the question "Do prototypes possess clinical features?" is **negative**. The detailed analysis is as follows:

- **Shortcut Learning Phenomenon:** Examination of the figures clearly shows that the PIP-Net model (and consequently the Phase 3 Black-box models), instead of learning lung parenchyma tissue, have learned non-medical or Confounding Features. The prototypes focus on incorrect items such as the following:
    - ❖ Surrounding space and image framing, indicating that the model has discovered "biases present in dataset collection" rather than the disease; this issue becomes more severe upon further examination of prototypes with lower scores, where in many cases the surrounding black space is selected.
    - ❖ Bones, clavicle, and arm (which are not sites of pulmonary infection).
    - ❖ And even radiology tags.
- **Practical Value:** From a diagnostic perspective, these prototypes lack value because a physician cannot confirm the disease based on them. However, the practical value of this model lies in **"AI Safety Assurance"**. This model clarified that the current dataset is not reliable for training clinical models; a reality that remained hidden in the complex Phase 3 models.

Technical Notes on Visual Outputs: The number of images present in the visualised_prototypes folder may exceed the number of final (Non-zero) prototypes reported in the tables. The reason for this is that the visualization script (vis_pipnet.py) is set with a very low Threshold to plot any partial activity of the prototypes as well. This is while in the final classification layer, the weight of many of these prototypes is practically zero or negligible. To resolve the aforementioned ambiguity and accurately identify the prototypes that play a role in the final decision-making, an analyzer code snippet was developed in the last cell of the notebook (Visualization & Results). This code, by directly examining the model weights (classification_layer), filters and sorts the truly active prototypes (with weight > 0.001). The output of this analysis is a suitable reference for determining the importance of prototypes in the quantitative analysis section, and furthermore, the development of this code is generalizable for any execution of the model.

Conclusion & Future Work:

Although the efforts of this phase to develop an interpretable model with direct clinical value did not yield the desired diagnostic result due to the biased nature of the dataset, the main achievement of this research extended beyond a simple classifier. By achieving a high accuracy of 92% and simultaneously revealing noise-based prototypes (such as image margins), the PIP-Net model highlighted a fundamental challenge in the application of AI in medicine: "the danger of blind reliance on numerical accuracy." This finding demonstrated that the stunning results (close to 99%) of the Black-Box models executed in Phase 3 were likely due to learning these very irrelevant features (Shortcut Learning) rather than disease diagnosis. Therefore, this project proved that Explainable AI (XAI) methods serve not only to build transparent models but also as a tool for "Data Quality Assurance" and "Verification" of complex model performance, and their use before the deployment of any intelligent system in sensitive clinical environments can be a necessity.

Dedicated to F.K.h, with love